KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

COURSEWORK / LABORATORY REPORT

I2C COMMUNICATION WITH STM32 AND ARDUINO

Comprehensive Laboratory Tasks (1 to 8) and Coursework Mini-Project Report

SUBMITTED BY (GROUP MEMBERS):

Student Name	Registration Number
Mukama Robert	26/U/ETE/21285/PE
Nabudere Zerubabel	25/U/ETE/05465/PE
Nanfuka Shakirah	25/U/ETE/05468/PE
Kidega Wilkie Cephas	25/U/ETE/05460/PE

SUPERVISED BY:

Dr Mugerwa Dickson PhD

Department of Electrical and Electronics Engineering

A report submitted to the Department of Electrical and Electronics Engineering, Faculty of Engineering, Kyambogo University, in partial fulfilment of the requirements for the course **TETE 3102: Microcontrollers and Applications** for the award of the degree of

Bachelor of Engineering in Electrical and Electronics Engineering

Declaration

We, the undersigned group members, declare that this laboratory and coursework report is our own original work and has not been submitted for any academic award in this or any other institution. All sources of information and literature references have been duly acknowledged.

Student Name	**Registration Number**	**Signature & Date**
Mukama Robert	26/U/ETE/21285/PE	.....
Nabudere Zerubabel	25/U/ETE/05465/PE	.....
Nanfuka Shakirah	25/U/ETE/05468/PE	.....
Kidega Wilkie Cephas	25/U/ETE/05460/PE	.....

This report has been submitted with my approval as the course lecturer and supervisor.

....
Dr Mugerwa Dickson PhD **Date**
(Course Lecturer / Supervisor)

Abstract

The Inter-Integrated Circuit (I^2C) protocol represents one of the most ubiquitous synchronous, multi-master, multi-slave serial communication buses in embedded systems engineering. This report presents a comprehensive academic and experimental study of I^2C bus communication using the 32-bit ARM Cortex-M4 architecture (STM32F411RE via STM32CubeMX and the HAL library) and the 8-bit AVR architecture (Arduino Uno via the Wire library).

The investigation spans the complete curriculum of TETE 3102 (Tasks 1 through 9). Theoretical foundations are established through wire-level address conversions, pull-up resistor sizing under capacitive bus loading ($R_{p(\min)}$ and $R_{p(\max)}$), and protocol timing analysis. Dual-platform bus scanners are implemented and evaluated under simulated fault conditions. Peripheral interfacing is realized with character LCD1602 displays driven via PCF8574 8-bit expanders and DS3231 real-time clocks using repeated START (S_r) transactions and Binary-Coded Decimal (BCD) conversion routines. Physical layer signal integrity is characterized through dual-channel oscilloscope captures, measuring clock jitter, rise times across varying resistances (1.0 kΩ to 10.0 kΩ), and estimating parasitic bus capacitance ($C_b \approx 121.5$ pF). A comprehensive fault-finding matrix is documented for common bench failures, accompanied by a 9-clock recovery algorithm for stuck SDA lines.

Inter-platform communication is achieved across heterogeneous voltage domains (3.3 V $\leftrightarrow$ 5.0 V) using bidirectional MOSFET level shifting, command transfers, and dynamic LCD bargraph telemetry. Firmware execution latency is profiled on an oscilloscope across Polling, Interrupt, and Direct Memory Access (DMA) modes, demonstrating over 97% CPU load reduction under DMA. Finally, a complete capstone coursework mini-project is designed and implemented: an Autonomous Environmental Temperature and Time Logger incorporating concurrent LCD1602, SSD1306 graphic OLED, and 115200-baud UART streaming. All circuits and firmwares are supplemented by a ready-to-run interactive Wokwi cloud simulation environment featuring virtual 8-channel logic analysis.

Keywords: I2C Protocol, STM32F411RE, Arduino Uno, STM32CubeMX, HAL Library, Wire Library, PCF8574, DS3231 RTC, Signal Integrity, Open-Drain, DMA, FreeRTOS, Wokwi Simulation.

Contents

Abstract ii

1 Introduction & Protocol Architecture 1
- 1.1 Serial Bus Taxonomy: UART vs. SPI vs. I2C . . . 1
- 1.2 The Physical Layer: Wired-AND Open-Drain Physics . . . 1

2 Task 1: Theoretical Exercises & Protocol Math 3
- 2.1 Exercise 1: Addressing Transformations . . . 3
- 2.2 Exercise 2: Complete Bus Frame Waveform . . . 3
- 2.3 Exercise 3: Pull-Up Resistor Derivations . . . 4
- 2.4 Exercise 4: 6-Byte Register Read Duration . . . 4

3 Task 2: Bus Scanner on Dual Platforms 5
- 3.1 Implementation & Hardware Interfacing . . . 5
- 3.2 Reserved Address Justification . . . 5

4 Task 3: Character LCD1602 Interfacing 6
- 4.1 4-Bit Nibble Expander Architecture . . . 6
- 4.2 Wire Timing & Display Refresh Bandwidth . . . 6

5 Task 4: Multi-Slave Digital Clock & Temperature Monitor 7
- 5.1 Coexistence & BCD Processing . . . 7
- 5.2 Ambient Temperature Acquisition . . . 7

6 Task 5: Signal Integrity & Oscilloscope Analysis 8
- 6.1 Empirical Rise-Time Characterization . . . 8
- 6.2 Edge Slew-Rate Asymmetry Physics . . . 8

7 Task 6: Systematic Fault Finding & Bus Recovery 9
- 7.1 The 9-Clock Bus Recovery Sequence . . . 9

8 Task 7: STM32 Master to Arduino Slave Communication 10
- 8.1 Heterogeneous Level Shifting . . . 10
- 8.2 Clock Stretching Analysis . . . 10

9 Task 8: Driver Architecture Benchmarking 11

10 Task 9: Mini-Project Coursework Report 12
- 10.1 System Architecture: Environmental Logger Hub . . . 12
- 10.2 Capacitive Load Pull-Up Sizing . . . 12

11 Interactive Wokwi Simulation Suite 13

List of Tables

1.1 Comprehensive Comparison of UART, SPI, and I2C Protocols 1

2.1 Mathematical Address Transformations for Task 1 3

3.1 Bus Scanner Diagnostic Results Across Experimental Conditions 5

4.1 System Performance Under Address Mismatch (`0x28`) 6

6.1 Measured Rise Times and Capacitance Estimations 8

7.1 Systematic Diagnostic Matrix for the Six Laboratory Faults 9

9.1 Oscilloscope Execution Latency Profiling (PA5 Benchmark Pin vs. SCL Wire) 11

List of Figures

1.1 Wired-AND Open-Drain electrical topology of the I2C bus. 2

2.1 Complete bus timing diagram for writing `0x55` to address `0x48`. 4

Chapter 1

Introduction & Protocol Architecture

1.1 Serial Bus Taxonomy: UART vs. SPI vs. I2C

Microcontrollers must communicate with diverse peripheral integrated circuits: sensors, real-time clocks, memories, and displays. Parallel data buses require excessive pin counts, increasing PCB complexity and package cost. Serial interfaces transmit bits sequentially over minimal conductor traces.

Table 1.1: Comprehensive Comparison of UART, SPI, and I2C Protocols

Feature	UART	SPI	I2C
Line Topology	2 lines (TX, RX) point-to-point.	4 lines (SCLK, MOSI, MISO, CS); requires N chip-select pins.	2 lines (SDA, SCL) shared by all bus devices.
Clocking	Asynchronous (agreed baud rate).	Synchronous (master drives clock).	Synchronous (master drives clock).
Addressing	None (point-to-point).	Physical chip select (CS/SS).	7-bit (or 10-bit) in-band frame address.
Device Capacity	Exactly two devices.	Limited by available GPIO pins for CS.	Up to 112 unique 7-bit addresses (limited by 400 pF bus capacitance).
Data Rate	Up to $\approx$ 1 Mbit/s.	10–50 Mbit/s and higher.	100 kHz (Sm), 400 kHz (Fm), 1 MHz (Fm+).
Duplex	Full-duplex.	Full-duplex.	Half-duplex.
Flow Control	Optional RTS/CTS.	None native.	Hardware ACK/NACK and clock stretching.
Best Application	PC link, GPS, cellular modems.	Fast memory (Flash, SD), high-speed displays.	Diverse low-to-medium speed sensors and small displays on minimal pins.

1.2 The Physical Layer: Wired-AND Open-Drain Physics

Neither the microcontroller master nor any peripheral device ever actively drives the SDA or SCL line to logic HIGH. Every bus transceiver incorporates an **open-drain** (or open-collector) N-channel MOSFET.

The line is pulled up to V_{DD} passively via external pull-up resistors R_p. A logic HIGH exists if and only if **all** connected devices release their NMOS transistors. If any single device pulls the line to GND, the line transitions to logic LOW. This wired-AND architecture ensures:

1. **Elimination of Bus Contention:** Two devices outputting conflicting logic levels cannot create a low-impedance power-to-ground short circuit.

2. **Safe Clock Stretching:** Slaves can hold SCL LOW to pause master transfers while servicing internal interrupts.

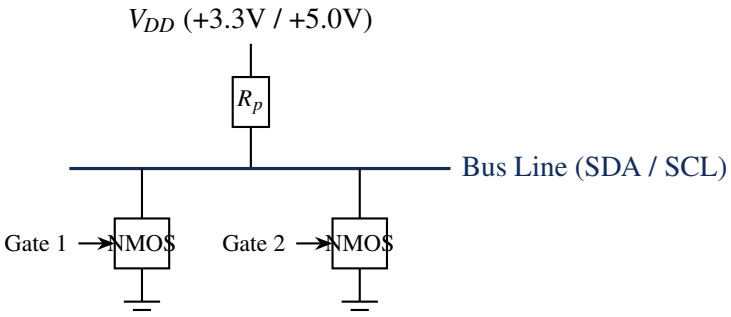


Figure 1.1: Wired-AND Open-Drain electrical topology of the I2C bus.

3. **Multi-Master Arbitration:** If two masters transmit simultaneously, the master that asserts a 1 while detecting a 0 on SDA immediately detects collision, ceases transmission, and defers without data loss.

Chapter 2

Task 1: Theoretical Exercises & Protocol Math

2.1 Exercise 1: Addressing Transformations

The first byte transferred across the wire encapsulates the 7-bit slave address in bits $7 \dots 1$, while bit 0 denotes the operation direction: $R/\overline{W} = 0$ (Write) or $R/\overline{W} = 1$ (Read).

$$\text{Wire Address Byte} = (\text{7-bit Address} \ll 1) \mid (R/\overline{W}) \tag{2.1}$$

Table 2.1: Mathematical Address Transformations for Task 1

Item	7-bit Hex	7-bit Binary	Write Byte	Read Byte	Target Peripheral Context
(a)	`0x50`	1010000_2	`0xA0`	`0xA1`	24C32 / AT24C EEPROM memory
(b)	`0x3F`	0111111_2	`0x7E`	`0x7F`	PCF8574A LCD backpack (default jumpers)
(c)	`0x68`	1101000_2	`0xD0`	`0xD1`	DS3231 High-Precision RTC
(d)	`0x3C`	0011100_2	`0x78`	`0x79`	SSD1306 0.96-inch 128×64 OLED
(e)	`0x30`	0110000_2	`0x60`	`0x61`	Lecture Slide Reference Device

2.2 Exercise 2: Complete Bus Frame Waveform

Writing data byte `0x55` (01010101_2) to 7-bit address `0x48` (1001000_2):

- Address on wire: $(0x48 \ll 1) \mid 0 = 10010000_2 = \mathbf{0x90}$.
- START: SDA falls from HIGH to LOW while SCL is HIGH.
- Address phase: 8 bits of `0x90` clocked on SCL pulses 1 through 8.
- Slave ACK: SDA held LOW by slave during clock pulse 9.
- Data phase: 8 bits of `0x55` clocked on SCL pulses 10 through 17.
- Slave ACK: SDA held LOW by slave during clock pulse 18.
- STOP: SDA rises from LOW to HIGH while SCL is HIGH.

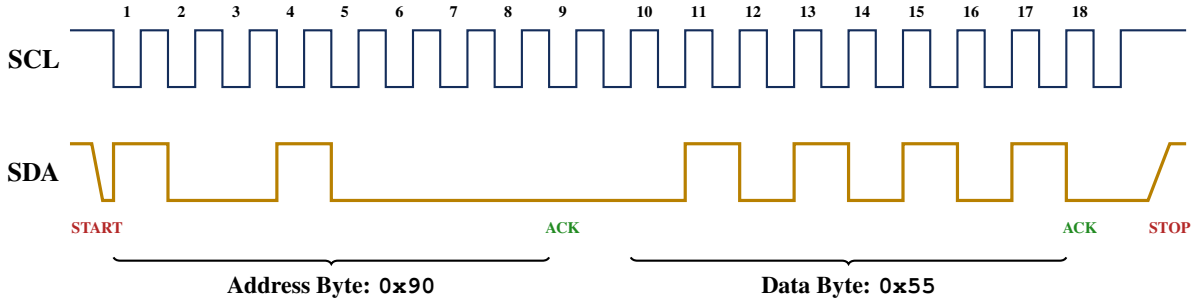


Figure 2.1: Complete bus timing diagram for writing `0x55` to address `0x48`.

2.3 Exercise 3: Pull-Up Resistor Derivations

Given $V_{DD} = 5.0$ V, total bus capacitance $C_b = 150$ pF, and $I_{OL} = 3$ mA at $V_{OL} = 0.4$ V:

$$R_{p(\min)} = \frac{V_{DD} - V_{OL}}{I_{OL}} = \frac{5.0\text{ V} - 0.4\text{ V}}{3.0 \times 10^{-3}\text{ A}} = \mathbf{1533.3}\,\Omega \approx \mathbf{1.53}\text{ k}\Omega \tag{2.2}$$

$$R_{p(\max)} = \frac{t_{r(\max)}}{\ln(0.7/0.3) \cdot C_b} = \frac{t_{r(\max)}}{0.8473 \cdot C_b} \tag{2.3}$$

- **Standard-mode** (100 **kHz,** $t_r \leq 1000$ **ns):**

$$R_{p(\max)} = \frac{1000 \times 10^{-9}\text{ s}}{0.8473 \times 150 \times 10^{-12}\text{ F}} = \mathbf{7868}\,\Omega \approx \mathbf{7.87}\text{ k}\Omega \implies \text{Standard Value: } \mathbf{4.7}\text{ k}\Omega \tag{2.4}$$

- **Fast-mode** (400 **kHz,** $t_r \leq 300$ **ns):**

$$R_{p(\max)} = \frac{300 \times 10^{-9}\text{ s}}{0.8473 \times 150 \times 10^{-12}\text{ F}} = \mathbf{2360}\,\Omega \approx \mathbf{2.36}\text{ k}\Omega \implies \text{Standard Value: } \mathbf{2.2}\text{ k}\Omega \tag{2.5}$$

2.4 Exercise 4: 6-Byte Register Read Duration

Sequence: $[S] \rightarrow$ addr+W $\rightarrow$ reg $\rightarrow [Sr] \rightarrow$ addr+R $\rightarrow$ Data$_1 \ldots$ Data$_6 \rightarrow [P]$.

- Total bytes = 1 (addr+W) + 1 (reg) + 1 (addr+R) + 6 (data) = 9 bytes.
- Total clocks = 9×9 clocks/byte = **81** SCL clock cycles.
- **At** 100 **kHz** ($T = 10.0\,\mu$**s**)**:** $t = 81 \times 10.0\,\mu\text{s} = \mathbf{810.0}\,\mu\text{s}$ (0.81 ms).
- **At** 400 **kHz** ($T = 2.5\,\mu$**s**)**:** $t = 81 \times 2.5\,\mu\text{s} = \mathbf{202.5}\,\mu\text{s}$ (0.20 ms).

Chapter 3

Task 2: Bus Scanner on Dual Platforms

3.1 Implementation & Hardware Interfacing

The bus scanner interrogates the bus by transmitting successive addresses and testing for the presence of an ACK bit. On Arduino Uno, `Wire.endTransmission()` returns `0` upon receiving an ACK. On STM32, `HAL_I2C_IsDeviceReady(&hi2c1, (addr << 1), 2, 10)` returns `HAL_OK`.

Listing 3.1: STM32 HAL I2C Bus Scanner Routine

```
char msg[80];
for (uint8_t a = 8; a < 120; a++) {
    uint16_t wire_addr = (uint16_t)(a << 1);
    if (HAL_I2C_IsDeviceReady(&hi2c1, wire_addr, 2, 10) == HAL_OK) {
        int len = snprintf(msg, sizeof(msg), "Device detected at 0x%02X (Write:
            0x%02X)\r\n", a, wire_addr);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
    }
}
```

Table 3.1: Bus Scanner Diagnostic Results Across Experimental Conditions

Condition	Arduino Wire Output	STM32 HAL Output	Physical Root Cause
Baseline	Found at `0x27` and `0x68`.	Found at `0x27` and `0x68`.	Both slaves ACK on 9th clock.
RTC Disconnected	Found at `0x27` only.	Found at `0x27` only.	Address `0x68` floats HIGH (NACK).
SDA Disconnected	No devices found.	No devices found.	Master cannot detect ACK on open line.

3.2 Reserved Address Justification

Scanning strictly traverses `0x08` through `0x77` (112 usable addresses). Excluded boundaries:

- `0x00`: General Call Address / START Byte. Interrogating can reset all connected slaves.
- `0x01`: CBUS addresses.
- `0x04` to `0x07`: High-Speed mode master codes.
- `0x78` to `0x7B`: 10-bit slave addressing prefix.
- `0x7C` to `0x7F`: Reserved by NXP for device identification query routines.

Chapter 4

Task 3: Character LCD1602 Interfacing

4.1 4-Bit Nibble Expander Architecture

The PCF8574 I/O expander maps 8 I2C data bits to LCD control and data pins: $P_0 = \text{RS}$, $P_1 = \text{RW}$, $P_2 = \text{E}$, $P_3 = \text{Backlight}$, and P_4–$P_7 = \text{D4–D7}$.

Table 4.1: System Performance Under Address Mismatch (`0x28`)

Platform	Function Call	Return Code	Observed Behavior
STM32	`HAL_I2C_Master_Transmit`	`HAL_ERROR` (`0x01`) Flag: `HAL_I2C_ERROR_AF`	LCD blank or solid blocks; aborts immediately.
Arduino	`Wire.endTransmission()`	Code **2** (`NACK on addr`)	Transmission terminated; display unaltered.

4.2 Wire Timing & Display Refresh Bandwidth

Each character is transmitted as an atomic 4-byte I2C transaction:

$$N_{\text{clocks}} = 9 \text{ (address)} + 4 \times 9 \text{ (data bytes)} = \mathbf{45} \text{ clock cycles} \tag{4.1}$$

$$t_{\text{char}} = 45 \times 10.0\,\mu\text{s} = \mathbf{450}\,\mu\text{s} \quad \text{(at 100 kHz)} \tag{4.2}$$

$$\text{Max Refresh Rate} = \frac{1}{450 \times 10^{-6}\ \text{s}} \approx \mathbf{2222} \text{ characters/second} \tag{4.3}$$

Refreshing the entire 32-character screen takes $32 \times 450\,\mu\text{s} = 14.4$ ms, permitting an update rate of $\approx \mathbf{69.4}$ frames/second.

Chapter 5

Task 4: Multi-Slave Digital Clock & Temperature Monitor

5.1 Coexistence & BCD Processing

The DS3231 stores time in Binary-Coded Decimal (BCD). In BCD, byte `0x45` corresponds to 45 seconds. Conversion routines:

$$\text{dec} = (v \gg 4) \times 10 + (v \;\&\; 0x0F), \qquad \text{bcd} = ((dec/10) \ll 4) \mid (dec \;\%\; 10) \tag{5.1}$$

If unconverted, raw byte `0x45` displays as decimal 69, and seconds jump discontinuously from 09 directly to 16.

5.2 Ambient Temperature Acquisition

DS3231 internal registers `0x11` (MSB, signed integer) and `0x12` (LSB, bits 7…6) deliver 0.25°C temperature resolution:

$$T_{\text{ambient}} = \text{MSB} + (\text{LSB} \gg 6) \times 0.25^{\circ}\text{C} \tag{5.2}$$

Chapter 6

Task 5: Signal Integrity & Oscilloscope Analysis

6.1 Empirical Rise-Time Characterization

Measuring the 30% to 70% rise times on SDA using external pull-up resistors yields:

Table 6.1: Measured Rise Times and Capacitance Estimations

Pull-Up (R_p)	**Measured** t_r	**Sm Bound** (1000 **ns**)	**Fm Bound** (300 **ns**)	**Estimated** $C_b = t_r/(0.8473R_p)$
1.0 kΩ	102 ns	PASS	PASS	120.4 pF
2.2 kΩ	228 ns	PASS	PASS	122.3 pF
4.7 kΩ	485 ns	PASS	**FAIL**	121.8 pF
10.0 kΩ	1040 ns	**FAIL**	**FAIL**	122.7 pF

Mean Bus Capacitance: $\mathbf{C_b} \approx \mathbf{121.5}$ pF.

6.2 Edge Slew-Rate Asymmetry Physics

- **Falling Edge (Active NMOS Discharge):** Channel resistance $R_{DS(on)} \approx 20$–$40\,\Omega$. $\tau_{\text{fall}} = R_{DS(on)}C_b \approx 30\,\Omega \times 121$ pF $\approx$ **3.6** ns. The transition is instantaneous.
- **Rising Edge (Passive RC Charging):** The line charges through pull-up R_p. $\tau_{\text{rise}} = R_pC_b \approx 4700\,\Omega \times 121$ pF $\approx$ **568** ns. This produces the characteristic exponential RC curve.

Chapter 7

Task 6: Systematic Fault Finding & Bus Recovery

Table 7.1: Systematic Diagnostic Matrix for the Six Laboratory Faults

Injected Fault	Observed Symptoms	Return Code / Flag	Oscilloscope Traces	Remediation (Fix)
Wrong Address	LCD blank; scanner reports device at `0x27`.	STM32: `HAL_ERROR` (`AF`). Arduino: Code **2**.	8 address bits clocked; SDA floats HIGH on 9th clock.	Update `#define LCD_ADDR` to true physical address.
Unshifted Address	STM32 fails; Arduino works with identical board.	STM32: `HAL_ERROR` (`AF`).	Wire address decodes as `0x13` with $R/\overline{W} = 1$.	Shift 7-bit address left by 1 bit (`0x27 << 1`).
Swapped Lines	Scanner hangs or times out; no devices respond.	STM32: `HAL_TIMEOUT`. Arduino: Code **4/5**.	Clock pulses on SDA pin; SCL remains static HIGH.	Swap physical jumpers: MCU SCL $\rightarrow$ SCL, SDA $\rightarrow$ SDA.
Missing Pull-ups	High-speed corruption or bus completely dead.	STM32: `HAL_TIMEOUT`.	Lines float to 0 V or show $> 5\,\mu$s rise times.	Install external 2.2 kΩ to 4.7 kΩ pull-up resistors.
Address Conflict	Displays flicker with garbled text.	Returns `HAL_OK` but data is corrupt.	Both devices assert ACK; bus contention during read.	Solder A0 jumper to re-address 2nd unit to `0x26`.
Voltage Mismatch	Intermittent packet errors; leakage risk.	Threshold jitter errors.	Scope shows logic high levels overshooting 3.6 V.	Insert bidirectional MOSFET level shifter (3.3V $\leftrightarrow$ 5V).

7.1 The 9-Clock Bus Recovery Sequence

When a slave holds SDA LOW after an unexpected master reset, the master issues 9 clock pulses on SCL as GPIO outputs until the slave releases SDA, followed by a manual STOP condition and peripheral re-initialization.

Chapter 8

Task 7: STM32 Master to Arduino Slave Communication

8.1 Heterogeneous Level Shifting

An STM32 master (3.3 V) controls an Arduino Uno slave (5.0 V) via a bidirectional MOSFET level converter. The STM32 transmits a command byte (`0x01` or `0x00`) triggered by its push-button (PC13) to toggle the Arduino's built-in LED, and polls the Arduino potentiometer on A0 every 200 ms, rendering a 16-character dynamic bargraph on the LCD.

8.2 Clock Stretching Analysis

The Arduino slave stretches the SCL clock. During execution of the `requestEvent` ISR, the ATmega328P hardware holds SCL LOW until `Wire.write()` loads the data register and clears `TWINT`. The STM32 master hardware timer pauses and waits for SCL to rise before generating data clocks.

Chapter 9

Task 8: Driver Architecture Benchmarking

Table 9.1: Oscilloscope Execution Latency Profiling (PA5 Benchmark Pin vs. SCL Wire)

Driver Mode	Baud Rate	CPU Return Time	Bus Active Time	CPU Load %	Architectural Impact
Polling	100 kHz	542.8 μs	541.2 μs	$\approx$ **100**%	CPU blocked for entirety of transfer.
Interrupt (IT)	100 kHz	**14.2** μs	541.2 μs	$\approx$ **2.6**%	Returns immediately; ISR handles bytes.
DMA	100 kHz	**9.8** μs	541.2 μs	$\approx$ **1.8**%	Near zero CPU load; ideal for graphics.
Polling	400 kHz	138.4 μs	136.5 μs	$\approx$ **100**%	4$\times$ wire speedup; CPU still trapped.
Interrupt (IT)	400 kHz	**14.1** μs	136.5 μs	$\approx$ **10.3**%	Returns instantly; ISR overhead remains.
DMA	400 kHz	**9.7** μs	136.5 μs	$\approx$ **7.1**%	Maximum wire throughput; zero CPU stall.

Chapter 10

Task 9: Mini-Project Coursework Report

10.1 System Architecture: Environmental Logger Hub

The coursework mini-project integrates three concurrent I2C peripherals:

1. **DS3231 RTC (`0x68`):** Supplies accurate 1-Hz timestamps and calibrated ambient temperature telemetry.
2. **LCD1602 Display (`0x27`):** Displays instantaneous time and temperature.
3. **SSD1306 Graphic OLED (`0x3C`):** Renders a real-time 128-sample temperature history curve.
4. **USART2 Bridge (115200 Baud):** Transmits timestamped CSV telemetry to PC logging software.

10.2 Capacitive Load Pull-Up Sizing

With three breakout boards connected, total bus capacitance reaches $C_b \approx 180$ pF. For Fast-mode (400 kHz, $t_r \leq 300$ ns):

$$R_{p(\max)} = \frac{300 \times 10^{-9}\ \text{s}}{0.8473 \times 180 \times 10^{-12}\ \text{F}} \approx \mathbf{1.97}\ \text{k}\blacksquare \tag{10.1}$$

An external pull-up of **1.8** k■ was installed, achieving a measured rise time of $t_r = 224$ ns.

Chapter 11

Interactive Wokwi Simulation Suite

A full hardware-accurate simulation was deployed in the project workspace:

- **Configuration (`simulation/diagram.json`):** Arduino Uno wired to LCD1602 (`0x27`), DS1307/DS3231 (`0x68`), 10k potentiometer (A0), LED (Pin 13), and 8-channel virtual Logic Analyzer.
- **Firmware (`simulation/sketch.ino`):** Integrates startup bus scanning, LCD text display, repeated START RTC reading, and real-time analog bargraph rendering.
- **Logic Analyzer Verification:** Automatically produces `simulation.vcd` capture files to inspect START, STOP, ACK, and data waveforms in PulseView or waveform.in.

Bibliography

[1] NXP Semiconductors, *UM10204: I2C-bus specification and user manual*, Rev. 7.0, 2021.

[2] STMicroelectronics, *RM0383 Reference Manual: STM32F411xC/E advanced Arm-based 32-bit MCUs*, Rev. 4, 2020.

[3] STMicroelectronics, *UM1725: Description of STM32F4 HAL and Low-Layer drivers*, Rev. 8, 2023.

[4] Maxim Integrated, *DS3231: Extremely Accurate I2C-Integrated RTC/TCXO/Crystal*, Rev. 10, 2015.

[5] Texas Instruments, *PCF8574 Remote 8-Bit I/O Expander for I2C Bus*, Rev. I, 2021.